Chapter 8: Analyzing the Language of Climate Change in the United States Congressional Record

[Needs introduction]

```
library(tmha.data)
library(text2vec)
library(tidyverse)
library(kableExtra)

data("congress_daily_climate_change_2000_2020")

congress_daily_climate_change_2000 <- congress_daily_climate_change_2000_2020 %>%
  filter(decade == "2000")

climate_change_text_2000 <- congress_daily_climate_change_2000$content

congress_daily_climate_change_2010 <- congress_daily_climate_change_2000_2020 %>%
  filter(decade == "2010")

climate_change_text_2010 <- congress_daily_climate_change_2000$content

congress_daily_climate_change_2020 <- congress_daily_climate_change_2000_2020 %>%
  filter(decade == "2020")

climate_change_text_2020 <- congress_daily_climate_change_2020$content
```

When a computer analyzes language, it cannot work with words directly. Instead, it represents each word as a list of numbers, called a vector. For example, instead of storing the word "climate" as text, the model might represent it like this:

climate → [0.21, -0.44, 0.88, 0.13, . . .]

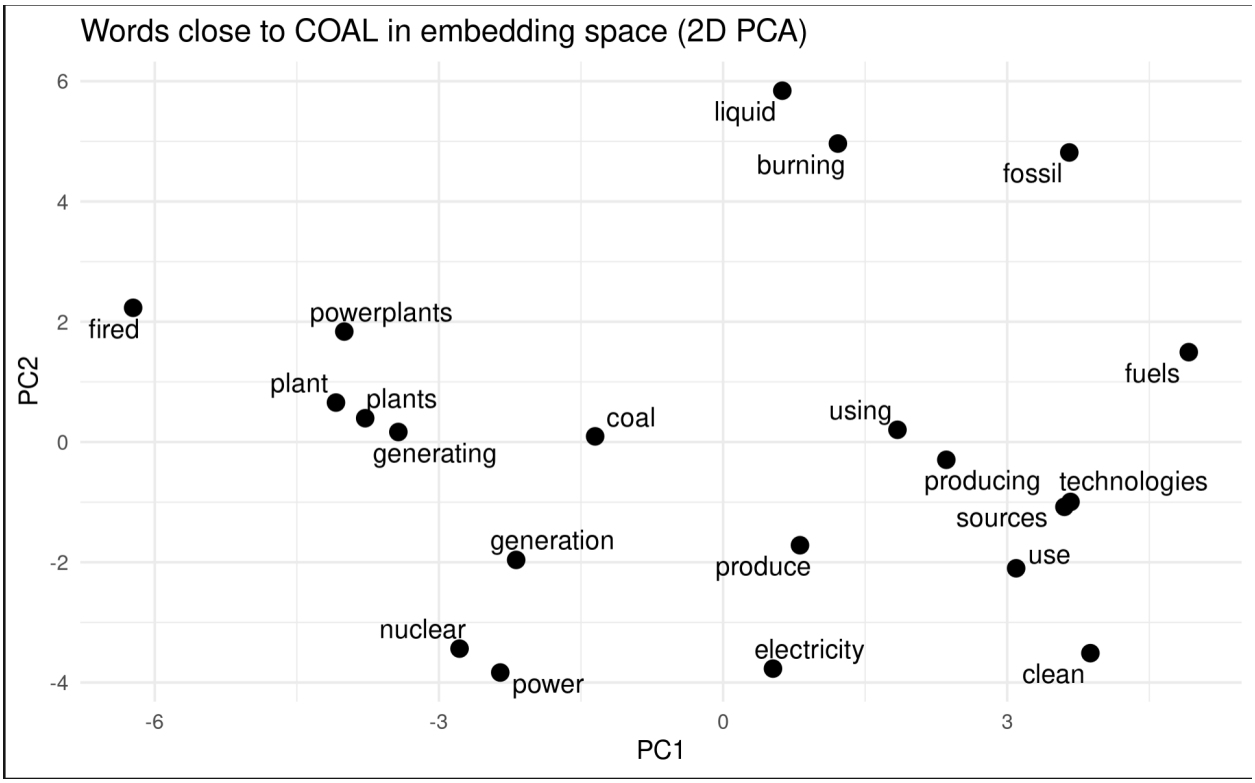
Each number in the vector captures a small piece of the word's meaning based on how the word appears next to other words in the corpus. By examining these patterns across large amounts of text, it becomes possible for a computer to estimate the numeric representations that reflect relationships between words. To do this, we will use a technique called machine learning.

Machine learning refers to computational methods that identify patterns in data and use those patterns to create numerical representations. These numerical represenations are improved through repeated calculations over the dataset. In this chapter, the specific machine learning approach used to represent words numerically is called word embeddings.

Word embeddings represent each word as a point in a numeric space. Words that appear in similar contexts tend to be placed near one another in this space. Because the words are represented numerically, the distance between them can be measured, allowing analysts to identify which words are most closely associated with a given term. The following image illustrates this idea. The word "fossil" appears closer to "fuels" than to

"nuclear." These associations are intuitive, reflecting the idea that fossil fuels commonly appear together in discussions of climate, whereas fossil is less association with nuclear, as two different forms of energy.

```
knitr::include_graphics("coal.png")
```



The following code demonstrates how to use a word embeddings model, GloVe.

```
# A function that trains GloVe word embeddings from a vector of text.
# The parameters control model size, context window, vocabulary filtering,
# training iterations, and reproducibility.
find_word_embeddings <- function(data,
                                 rank=50,
                                 window=5L,
                                 term_count_min=1,
                                 x_max=10,
                                 n_iter=30,
                                 seed=42) {

  # Tokenize the text: convert to lowercase and split sentences into words
  tokens <- word_tokenizer(str_to_lower(data))

  # Create an iterator over the tokenized text for efficient processing
  it <- itoken(tokens, ids = seq_along(tokens), progressbar = FALSE)

  # Build the vocabulary from the tokens and remove very rare words
  vocab <- create_vocabulary(it) %>%
    prune_vocabulary(term_count_min = term_count_min)

  # Convert the vocabulary into a vectorizer that maps words to indices
```

```r
  vectorizer <- vocab_vectorizer(vocab)

  # Create a term-co-occurrence matrix showing which words appear near each other
  # within the specified window size
  tcm <- create_tcm(it, vectorizer, skip_grams_window = window)

  # Set the random seed so the results can be reproduced
  set.seed(seed)

  # Initialize the GloVe model with the chosen dimensionality and weighting parameter
  glove <- GlobalVectors$new(rank = rank, x_max = x_max)

  # Train the model on the co-occurrence matrix for the specified number of iterations
  wv_main <- glove$fit_transform(tcm, n_iter = n_iter)

  # Extract the context vectors learned during training
  wv_context <- glove$components

  # Combine the main and context vectors to produce the final word embeddings
  word_embeddings <- wv_main + t(wv_context)
}
```

These settings control how the GloVe model learns relationships between words.

The rank determines how many numbers are used to represent each word. Representing words using more numbers allows the model to detect more subtle differences between words, while fewer numbers make the model simpler and faster but less precise.

The window (5) tells the model how many nearby words to pay attention to when learning meaning. The basic idea is that words that appear close together in sentences are often related, so the model looks at the five words on either side of each word.

The term_count_min (1) setting decides which words are included in the analysis. Words that appear fewer times than this threshold are ignored. This helps prevent extremely rare words, spelling mistakes, or noise in the data from influencing the model.

The x_max (10) setting reduces the influence of very common words. Words like "the," "and," or "of" appear so frequently that they could otherwise dominate the training process, so this parameter limits how much weight they receive.

The n_iter (30) parameter controls how many times the model goes through the data to refine its understanding of word relationships. More iterations usually improve the results, but they also make the process take longer.

Finally, the seed (42) ensures that the model produces the same results every time it runs with the same data and settings. This is important for reproducibility, especially in research.

GloVe prints short progress messages so it is possible to see how training is progressing. These messages include three pieces of information: the time, how many times the model has gone through the dataset (the epoch), and a number that indicates how well the model is learning (the loss value).

Model training happens iteratively. The term epoch refers to one complete pass through the entire dataset. This iteration is necessary because the model cannot capture the patterns in the data in a single pass. Instead, the training procedure gradually adjusts the model's parameters each time the data is processed. If the console shows epoch 10, it means the training process has gone through the entire corpus ten times, allowing the model to better capture statistical relationships between words.

The output also includes a loss value, which is a number that measures how well the model's current predictions match the patterns observed in the data. During training, the model compares the relationships

between words it predicts with the relationships that actually appear in the corpus. The loss value summarizes how different these two are. As training continues and the model's parameters are updated, the loss typically becomes smaller, indicating that the predictions are aligning more closely with the patterns in the text.

During training, messages such as INFO [] epoch , loss appear in the console. These updates allow the user to watch the model's progress as it repeatedly studies the corpus and gradually improves its understanding of relationships between words.

```
word_embeddings_2000 <- find_word_embeddings(climate_change_text_2000)
word_embeddings_2010 <- find_word_embeddings(climate_change_text_2010)
word_embeddings_2020 <- find_word_embeddings(climate_change_text_2020)
```

Finding Words with the Greatest Association

We can now write a function

```
get_similar <- function(keyword, word_embeddings, top_n = top_n_default) {

  kw_vec <- word_embeddings[keyword, , drop = FALSE]
  cos_sim <- sim2(word_embeddings, kw_vec, method = "cosine", norm = "l2")[, 1]

  tibble(word = names(cos_sim), similarity = unname(cos_sim)) %>%
    filter(word != keyword) %>%
    arrange(desc(similarity)) %>%
    slice_head(n = top_n) }
```

```
most_similar_climate <- get_similar("climate", word_embeddings_2000, 10)
most_similar_emissions <- get_similar("emissions", word_embeddings_2000, 10)
most_similar_coal <- get_similar("coal", word_embeddings_2000, 20)
```

```
tbl_2000 <- most_similar_climate %>%
  mutate(similarity = round(similarity, 3)) %>%
  kable(format = "latex",
    caption = 'Words Most Related to "Climate": 2000s Congressional Records',
        col.names = c("Word", "Cosine similarity"),
        align = c("l", "r"),
    booktabs = TRUE) %>%
  kable_styling(full_width = FALSE)

tbl_2000
```

```
tbl_2010 <- most_similar_emissions %>%
  mutate(similarity = round(similarity, 3)) %>%
  kable(format = "latex", caption = 'Words Most Related to "Emissions": 2010s Congressional Records',
    col.names = c("Word", "Cosine similarity"),
    align = c("l", "r"),
    booktabs = TRUE) %>%
  kable_styling(full_width = FALSE)

tbl_2010
```

Table 8.1: Words Most Related to "Climate": 2000s Congressional Records

Word	Cosine similarity
change	0.985
global	0.890
warming	0.836
address	0.725
impacts	0.715
issue	0.696
addressing	0.692
problem	0.671
policy	0.668
deal	0.652

Table 8.2: Words Most Related to "Emissions": 2010s Congressional Records

Word	Cosine similarity
greenhouse	0.916
carbon	0.856
dioxide	0.837
reduce	0.827
gas	0.796
gases	0.792
reductions	0.762
pollution	0.742
reducing	0.739
co2	0.730

```r
table_2020 <- most_similar_coal %>%
  mutate(similarity = round(similarity, 3)) %>%
  kable(format = "latex",
        caption = 'Words Most Related to "Coal": 2020s Congressional Records',
        col.names = c("Word", "Cosine similarity"),
        align = c("l", "r"),
        booktabs = TRUE) %>%
  kable_styling(full_width = FALSE,
                latex_options = c("hold_position"))

table_2020
```

Table 8.3: Words Most Related to "Coal": 2020s Congressional Records

Word	Cosine similarity
plants	0.834
power	0.808
using	0.747
burning	0.743
generation	0.741
fired	0.735
nuclear	0.726
use	0.697
powerplants	0.695
sources	0.689
produce	0.683
generating	0.662
fossil	0.659
fuels	0.658
electricity	0.655
liquid	0.654
plant	0.650
clean	0.650
producing	0.639
technologies	0.636

Vector Subtraction

Vector subtraction in word embeddings asks: what part of the meaning belongs to word1 but not word2? Because embeddings place words inside a multidimensional geometric "meaning space," where similar words live closer together, subtracting one vector from another isolates the semantic direction unique to the first term. Vector subtraction lets us reason about conceptual differences directly in mathematical space — extracting the portion of meaning that distinguishes one concept from another.

```r
vector_subtraction <- function(word1, word2, embeddings, top_n = 10) {

  # keep 1×d matrices so sim2() gets matrices on both sides
  a <- embeddings[word1, , drop = FALSE]
  b <- embeddings[word2, , drop = FALSE]
```

```r
  target <- a - b

  sims <- sim2(embeddings, target, method = "cosine", norm = "l2")[, 1]

  tibble(word = names(sims), similarity = unname(sims)) %>%
    filter(!word %in% c(word1, word2)) %>%
    arrange(desc(similarity)) %>%
    slice_head(n = top_n) }
```

```r
vector_subtraction("renewable", "fossil", word_embeddings_2000)
```

```
## # A tibble: 10 x 2
##    word          similarity
##    <chr>              <dbl>
##  1 portfolio          0.573
##  2 opportunities      0.530
##  3 creation           0.518
##  4 investment         0.517
##  5 frightfully        0.511
##  6 establishing       0.500
##  7 development        0.487
##  8 provides           0.484
##  9 incentives         0.479
## 10 program            0.470
```

If a query returns an error like:

```
Error in embeddings[word2, , drop = FALSE] : subscript out of bounds
```

then that means the word was not in the dataset.

```r
vector_subtraction("renewable", "fossil", word_embeddings_2010)
```

```
## # A tibble: 10 x 2
##    word          similarity
##    <chr>              <dbl>
##  1 creation           0.557
##  2 package            0.554
##  3 incentives         0.543
##  4 investment         0.532
##  5 portfolio          0.529
##  6 provides           0.515
##  7 opportunities      0.511
##  8 includes           0.489
##  9 establishing       0.488
## 10 development        0.486
```

Vector Addition

```r
vector_addition <- function(word1, word2, embeddings, top_n = 10) {
  a <- embeddings[word1, , drop = FALSE]
  b <- embeddings[word2, , drop = FALSE]

  target <- a + b

  sims <- sim2(embeddings, target, method = "cosine", norm = "l2")[,1]

  tibble(word = names(sims), similarity = unname(sims)) %>%
    filter(!word %in% c(word1, word2)) %>%
    arrange(desc(similarity)) %>%
    slice_head(n = top_n) }
```

If one of the words does not exist in the corpus, we will see the error `Error in FUN(left, right) : non-numeric argument to binary operator`.

```r
vector_addition("man", "woman", congress_daily_climate_change_2000)
```

```
## Error in FUN(left, right): non-numeric argument to binary operator
```

Word Association and the Congress Viewer

For the purposes of this book, we work with a limited subset of the data: three decades of Congressional Records. This smaller window allows analysts to run the data on a personal computer while still providing the tools to demonstrate the research strategies used throughout the book.

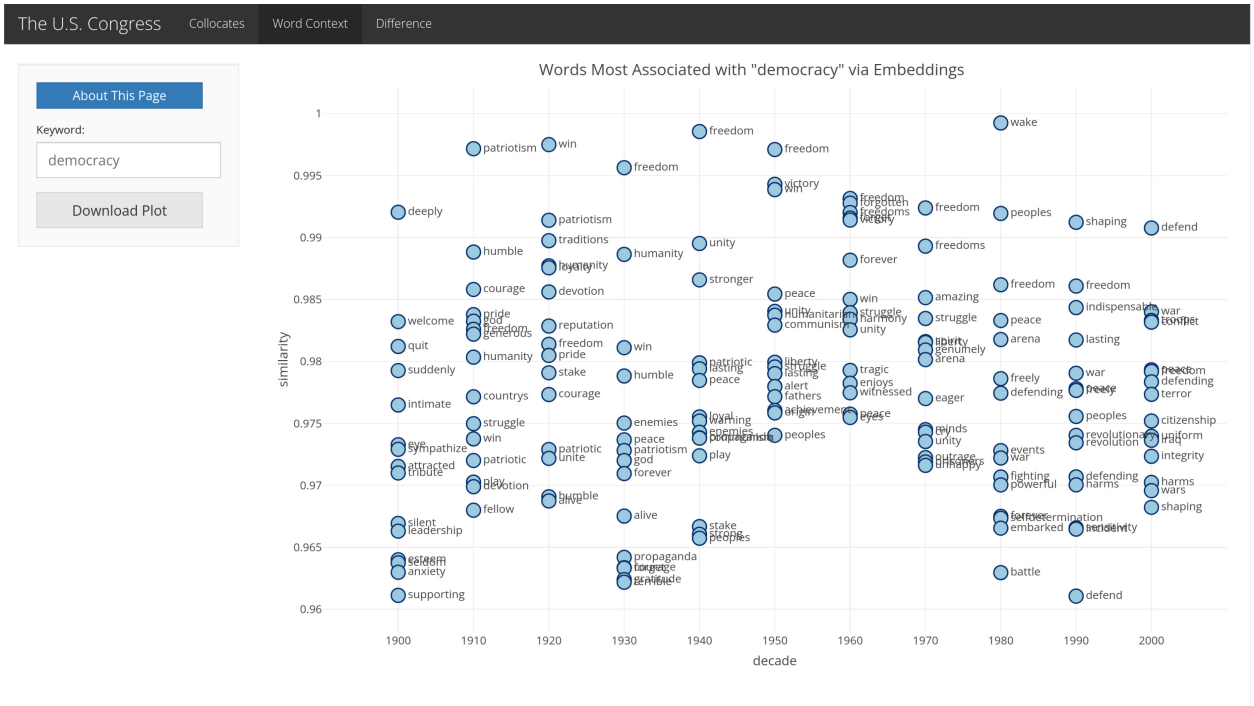
For researchers interested in working with larger collections, one of the authors of this book has also developed a web application called "Congress Viewer" (see: https://shinyviz.smu.edu/shiny/public/congress-viewer-demo/, navigate to the tab "Word Context"). The application provides access to the Stanford Corpus of the Congressional Record, allowing users to explore debates spanning 1900 to 2000 without needing to download or process the full dataset.

Apps like Congress Viewer are helpful for digital history research for several reasons. They support rapid iteration, where researchers can quickly test ideas by searching for different words and immediately seeing the results. Such accessibility allows analysts to move fluidly between hypothesis and exploration. Because these steps can be performed quickly through an interface, the researcher can perform much more exploration than would be practical if every step required rewriting and rerunning code to process large datasets.

Consider the visualizations generated using the Congress Viewer. The first visualization examines the word "democracy." For most of the twentieth century, the words most closely associated with democracy are fairly unsurprising: terms such as "patriotism," "freedom," "liberty," "victory," and "win." These associations largely carry a positive tone, and many are connected to words like "unity" and "peace."

By 2000, however, the pattern of associations shifts. During the period surrounding the Iraq War, the words most closely associated with "democracy" include "defend" (as the most associated word), "war," "troops," and "terror." The language surrounding democracy becomes suddenly tied to military action, reflecting the political language of the period.

```r
knitr::include_graphics("democracy.jpg")
```



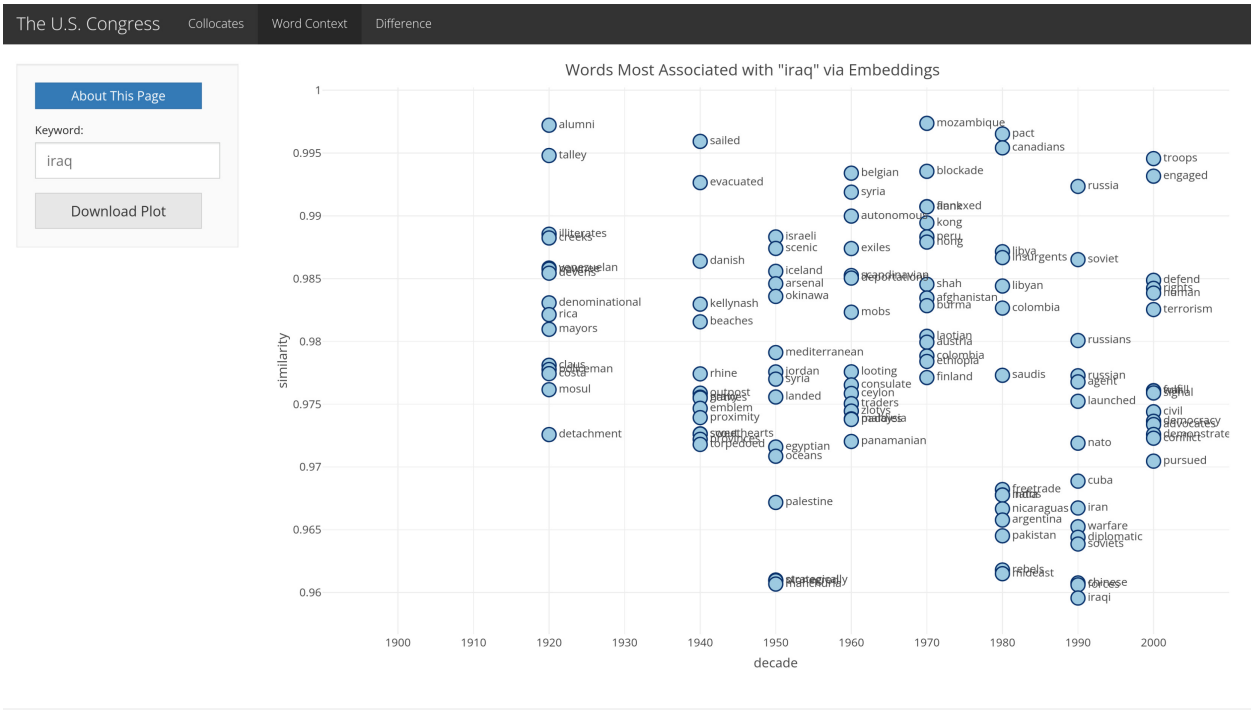
Now, cross-reference the democracy visualization with the word "iraq." Before 1920, Iraq appears too rarely in the corpus to appear in our word embeddings dataset. In the 1920s, the few associations that do emerge include Talley (also spelled Tallil)—a military base in Iraq—and Mosul, a major city in Iraq. Beyond these geographic references, there are few strong or consistent associations.

A similar pattern appears in 1940. The words most closely associated with "iraq" include "danish," "beaches," and "rhine." At first glance, these terms seem unrelated to Iraq. The most plausible explanation is data sparsity: because Iraq is mentioned in relatively few sentences in the Congressional Records, our statistical word embeddings model builds its associations based on a small set of debates. In that situation, any nearby geopolitical terms can highly influence associations, even if they are not conceptually related to Iraq. Therefore, Iraq's association with "danish," "beaches," or "rhine" does not suggest that it had any direct political relationship with the events taking place in Denmark or along the Rhine, such as military invasions.

In the 1950s, Iraq is most associated with other Middle Eastern nations such as Israel, Palestine, Egypt, Syria, and Jordan. These unremarkable associations stay consistent until the 1990s, where the rhetoric of post-9/11 emerges from the data. Iraq becomes more associated with Russia, an enemy to the United States, and "warfare." By 2000, the vocabulary surrounding Iraq shows clear signs of the emerging conflict that would become the Iraq War. Words such as "troops," "defend," and "democracy" appear more frequently in association with Iraq. Around this time, the term terrorism also begins to appear, reflecting the growing rhetorical framing that would soon develop into the broader discourse of the "War on Terror."

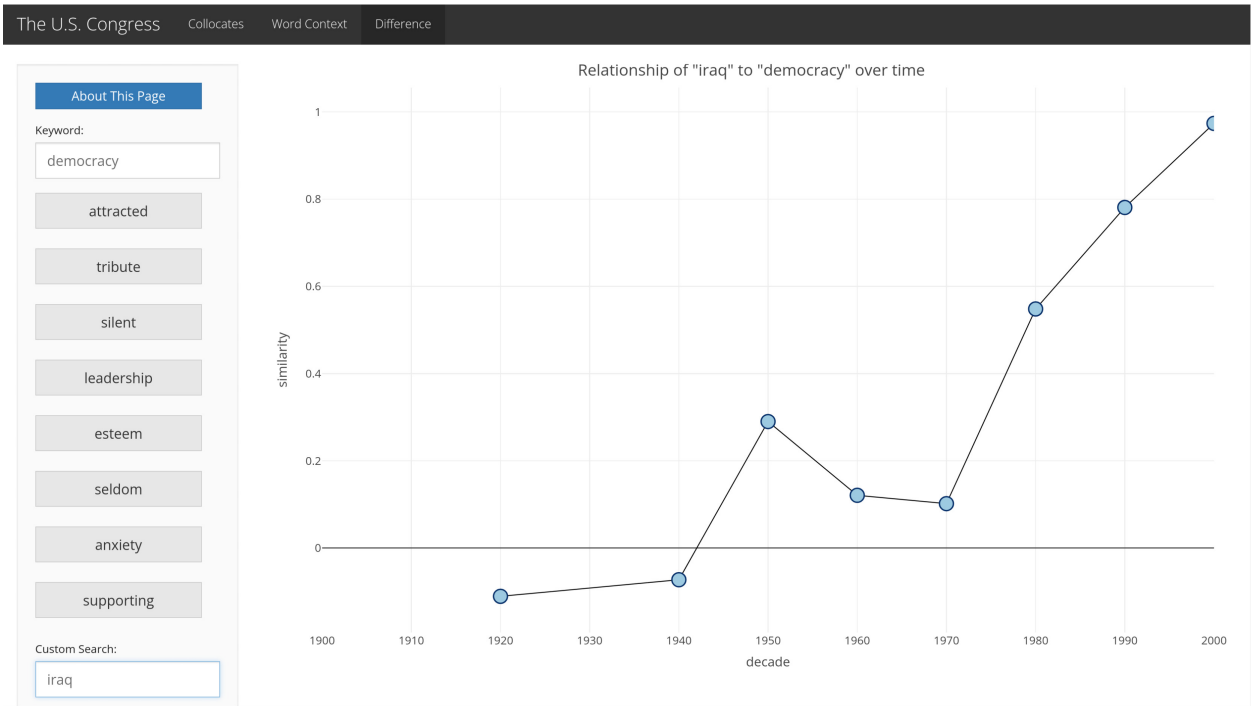
Perhaps most interesting is that we have measured the extent to which the words associated with "Iraq" dramatically become aligned with the words most associated with "Democracy." Comparing this visualization with one generated for the word "Iraq" suggests that discussions of democracy and Iraq became increasingly intertwined. In other words, the rhetoric of democracy in congressional debate appears to have been reframed in 2000, with the promotion or defense of democracy used as a justification for military action in Iraq.

```
knitr::include_graphics("iraq.jpg")
```



Analyzing Association Between Two Words Over Time using Congress Viewer

```
knitr::include_graphics("democracy_to_iraq.jpg")
```



Exercises

1. Word embeddings reveal patterns of association in language, but they are only one analytical lens. What other historical research methods could be paired with this approach to deepen interpretation? Explain how combining at least two methods could produce a richer understanding of climate discourse in Congressional records.

2. Select one word strongly associated with "climate" (for example: change, global, or warming). Then, locate one or two speeches or documents from the Congressional Record containing that word. Compare your close reading of these records with the computational result. Does the statistical association match the context of the Congressional Records?

3. In "Vectors Over Time," we analyze two words and their growing or lessing association with one-another. Examine the cosine similarity trajectory between two terms over a chosen range of time (for example, coal and emissions). What historical events might correspond to rises or declines in similarity? How could this quantitative signal guide further research?

4. Choose a decade and examine which climate-related terms are missing or unusually rare. Are there words you would expect but do not see? Does silence itself signal political positioning? How might archival absence complicate computational interpretation? Write a short reflection on how historians might interpret silence differently than data scientists based on their training.

5. Come up with five words that you think are signals of debates on climate (for example: regulation, China, hurricane, jobs). For each word, write code and use the `tmha.data` to compute the similarity of the word to climate. Which decade shows the strongest link to climate for each word? What's your speculation for why this association might exist?

6. Use Congress Viewer to search for your own words. Do you notice any trends when you compare multiple words side-by-side? For example, in the chapter we compared "democracy" and "Iraq" and saw that by the early 2000s the language around democracy increasingly included words like war, troops, and defend. Do similar patterns appear in the words you choose? Write a few sentences describing what you observe and what historical events might explain any word associations.

7. Like the example in this chapter—where "Iraq" appears associated with words such as "Danish" or "Rhine"—word embeddings sometimes produce associations that seem surprising or historically unrelated. Find another example in Congress Viewer where a word is associated with terms that seem unexpected. Why might these unexpected associations occur in the data? Could the result be explained by politician's rhetorical framing, or by sparse data?